

GlobeTrotter

Distributed Systems Questions & Answers

Technical discussion guide based on the app architecture and community-room experience

Scope

This document focuses on the design and operation of the application as a distributed system. It discusses service boundaries, routing, consistency, caching, real-time communication, presence, pagination, calls, resilience, security, observability, and storage trade-offs. It intentionally avoids code-level interview questions.

Structure

Part I contains questions only. Part II contains the corresponding answers in the same order, so the first question maps to the first answer, and so on.

Prepared from the repository architecture, deployment configuration, and real-time community-room behavior.

Part I ? Questions

Use this section for a technical interview, design review, or project defense. Try to answer each question before consulting Part II.

- 1. What role does the API gateway play in the GlobeTrotter architecture, and why is it useful when the backend is split into services?**
- 2. How would you describe the responsibility boundary between auth-service and data-service?**
- 3. Why does the application use a cache-first or weak-connection strategy for reads, but not for every write?**
- 4. What consistency model is appropriate for the community room, and why is strong global consistency not required for every screen update?**
- 5. How does the community room keep presence information reasonably fresh in a distributed environment?**
- 6. Why does the chat history use cursor-based pagination instead of downloading the complete room history?**
- 7. What distributed-systems problem occurs when a new message arrives while the user is reading older messages, and how should the UI respond?**
- 8. Why is a WebSocket useful for events and chat invalidations, and what should happen when that connection drops?**
- 9. How can the system prevent two users or two backend instances from starting conflicting shared call sessions in the same community room?**
- 10. What failure modes should be considered when the gateway can reach one microservice but not the other?**
- 11. How should authentication and authorization work across the gateway and the two services?**
- 12. What observability would help diagnose a delayed message, failed itinerary write, or stale presence indicator?**

13. What are the trade-offs of using shared file-backed data volumes in a multi-service deployment?

14. How should client-side cached images and API data be invalidated when a destination, post, or profile changes?

Part II ? Answers

The answers explain the architectural reasoning and the distributed-systems concepts that apply to GlobeTrotter. They describe intended design behavior and trade-offs rather than reproducing source code.

1. Answer

The API gateway is the stable public entry point for the Flutter client. It routes authentication-related paths to the auth-service, product and social paths to the data-service, and proxies the events WebSocket. This hides internal service addresses from clients, centralizes cross-cutting concerns such as routing and connection policy, and allows services to be deployed or scaled independently without changing the mobile app's API base URL.

2. Answer

The auth-service owns identity operations such as registration, login, email verification, approval flows, profiles, and token creation. The data-service owns travel and collaboration features such as destinations, itineraries, recommendations, posts, chat, events, statistics, assistant requests, and calls. This separation reduces coupling and limits which service needs to handle sensitive identity state.

3. Answer

Cached reads can safely provide a recent view of destinations, feeds, or previously loaded content while connectivity is poor. Writes such as posting, itinerary changes, presence heartbeats, and call operations have side effects and require server confirmation. Blindly retrying them can create duplicates, stale updates, or incorrect presence, so the app requires connectivity for mutations and calls while refreshing cached data in the background.

4. Answer

The community room can use eventual consistency for ordinary message lists and presence indicators. Small delays are acceptable as long as messages converge and the interface clearly indicates new activity. Stronger coordination is needed for operations such as call-session ownership or authenticated writes, where conflicting decisions would be visible or harmful. The design therefore uses different consistency expectations for different data types.

5. Answer

The client sends a heartbeat periodically, described in the implementation as a five-second heartbeat, and requests or receives refreshed presence data. A server can treat a user as active only while recent heartbeats exist and can expire stale records after a timeout. This is a lease-like approach: presence is temporary and must be renewed rather than stored as a permanent boolean.

6. Answer

A room can grow without bound, so downloading all messages increases latency, memory use, bandwidth, and rendering work. The client loads a bounded first page and requests older pages using a cursor such as the oldest loaded message. Cursor pagination is more stable than page numbers when new messages are inserted at the front, because the cursor identifies a position in the message stream rather than an offset that can shift.

7. Answer

The client must merge two timelines without losing messages or disrupting the user's reading position. New messages should be inserted in the correct order, while the scroll position for older content should be preserved. If the user is away from the newest position, the UI can show a new-message indicator instead of forcing a jump. This separates data convergence from presentation behavior.

8. Answer

A WebSocket allows the server to push compact invalidation or update signals without polling every screen continuously. The client should treat the socket as a best-effort live channel: detect disconnection, reconnect with backoff, resubscribe to relevant state, and fall back to ordinary HTTP refreshes. Reconnection must be idempotent so repeated connection attempts do not create duplicate listeners or inconsistent local state.

9. Answer

Call creation should be decided by an authoritative server-side coordination mechanism rather than by local UI state. A conditional write, lease, or transaction can reserve the room's active call slot, include an expiry, and release it when the call ends. If two requests race, only one reservation succeeds and the other receives a clear conflict response. This is an example of needing stronger coordination than ordinary presence updates.

10. Answer

The gateway should distinguish upstream timeouts, connection failures, and valid application errors. It should return explicit, safe error responses, apply bounded timeouts, and avoid retrying non-idempotent operations automatically. Health checks and logs should identify which upstream is unavailable. The client can continue showing cached read data where appropriate, but it should not present a failed write or call as successful.

11. Answer

The client presents an access token to the gateway, and the request identity must be validated before protected operations proceed. The gateway may forward the token, while each service should still enforce authorization for resources it owns rather than trusting a client-supplied user ID. Token expiry, invalid signatures, and insufficient permissions should produce explicit failures. Service-to-service communication also needs trusted configuration and protected credentials.

12. Answer

Use structured logs with a correlation or request ID propagated from the gateway to downstream services. Record service name, route, user or room context where safe, latency, status, and upstream failure details. Metrics should cover gateway latency, WebSocket reconnects, heartbeat expiry, message processing delay, cache age, and call reservation conflicts. Traces are especially useful for following one request across the gateway and services.

13. Answer

A shared volume is simple for a prototype and can let services see common JSON-backed state, but it is not a robust distributed database. Concurrent writes can race, file locking and crash recovery are difficult, and horizontal replicas may not share a consistent view unless the storage layer provides those guarantees. As scale or correctness requirements increase, service-owned durable storage with transactions and explicit event or API boundaries is safer.

14. Answer

The system should use explicit freshness rules and versioned or keyed cache entries. A server event can invalidate or mark a resource stale, after which the client re-fetches it. Time-to-live alone is simpler but can show stale content longer; event-driven invalidation improves freshness but requires reconnect and missed-event recovery. The client should also isolate cached data by API environment and account so users do not see another account's data.